Agtron points out that the solution in listing 11.6 works fine if data is being received frequently. If it's not being received frequently, you need to consider buffering and interpolation. That, however, is a lesson for another day. The final thing to consider with your browser-based program is how to put it together.

11.6. Structuring programs

Depending on the nature or circumstances of the problem (or the people solving the problem), an immediate-mode API can be inappropriate. Suppose that you must work with `canvas`, but you really need a retained-mode API. What can you do? Use the powerful design technique called wishful thinking.

11.6.1. Abstraction and APIs

Write your program *as if you already had the retained-mode API* that you want, and then implement it. Suppose you want a fluent retained-mode API that can draw a circle and then move the circle around. You call the API `Cézanne` after the famous French artist Paul Cézanne:

```
scene = Cézanne
.createScene('#scene')
.size(400, 400)

circle = scene
.createCircle()
.radius(10)
.color(Cézanne.RawUmber)
.position(20, 20)

circle.animatePosition 360, 360, 2
```

This looks *very* different from the `canvas` API. If you need an API like `Cézanne` but you start with raw `canvas`, then every single line that you write moves you farther from the API that you want. It's tempting to think that because you're not writing a library for other people to use, you don't need to create a nice, readable API. The opposite is true. Unless your